



Testing

Given a customer testing strategy, describe the appropriate test execution methodology and coverage requirements.

Table of Contents

-  [Test Execution Methodology](#)
-  [Apex Test Execution](#)
-  [Code Coverage](#)
-  [Testing Lightning Components](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the considerations for defining an appropriate test execution methodology.
-  Identify the options and considerations for executing Apex unit tests.
-  Describe the code coverage requirements that are applicable to the execution of Apex unit tests.
-  Describe how to test Lightning web components and Aura components.
-  Describe the appropriate test execution methodology and coverage requirements for the given customer requirements.



Introduction

In Salesforce, a **test execution methodology** is essential for a structured approach to executing tests, ensuring comprehensive coverage and alignment with best practices. Various **tools** are available for the **execution of Apex unit tests**, including Apex Classes page, Developer Console, and VS Code.

When using **Metadata API** for a deployment, a **subset of tests** can be run using the **RunSpecifiedTests** test level. Using a **mock object** is necessary to test Apex code that performs **HTTP callouts**. The **startTest()** and **stopTest()** methods can be used to **reset governor limits** in Apex unit tests.

For testing Lightning components, the **Jest** framework can be utilized. **Aura component accessibility tests** can be run in two environments: **JavaScript** and **WebDriver**



Test Execution Methodology & Coverage Requirements

Test Execution Methodology

A test execution methodology should focus on aspects such as a test plan, types of testing, test scripts, code coverage, automation, monitoring, and stakeholder engagement.

Apex Test Execution

Various options and considerations are applicable to the execution of Apex unit tests. For instance, a subset of tests can be executed during a deployment using the RunSpecifiedTests test level.



Code Coverage

Code coverage numbers are generated every time one or more tests are executed. At least 75% of Apex code must be covered by unit tests for a successful deployment.

Testing Lightning Components

Lightning web components and Aura components can be tested using the Jest framework. Apex controller can be tested using test classes. Aura accessibility tests can be run in JavaScript and WebDriver.

Test Execution Methodology

[Table of Contents](#) 

Test Execution Methodology

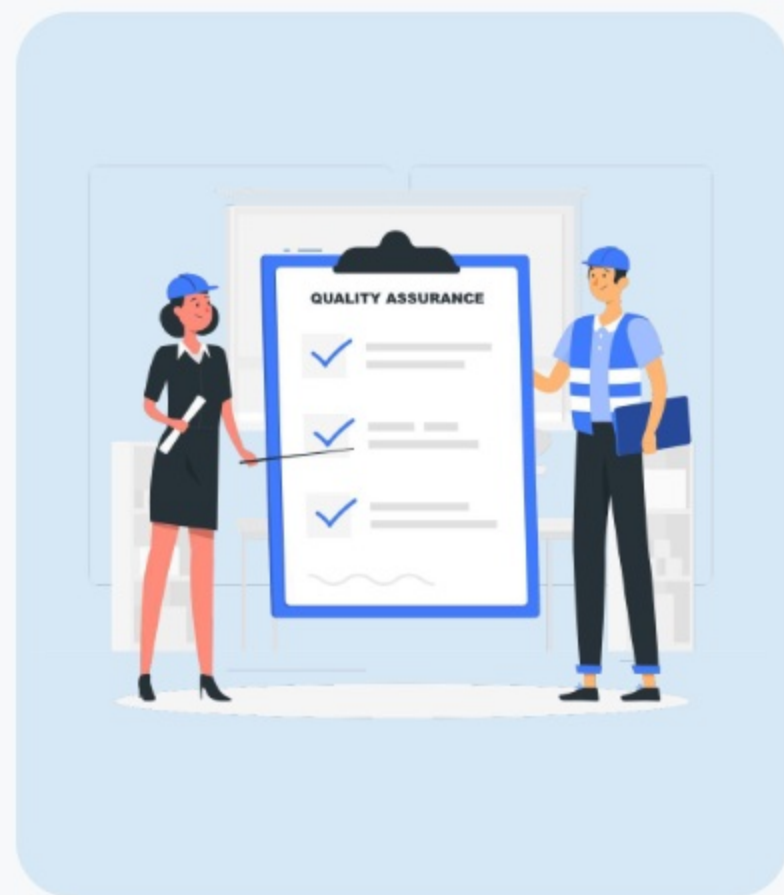
A test execution methodology should emphasize a **structured approach** to executing tests within Salesforce environments, ensuring **comprehensive coverage** and **alignment with best practices**.

❖ TEST PLAN

A test execution methodology should be based on defining a **clear test plan**, which should align with the customer's **business processes**, Salesforce **configuration**, and **customization**.

❖ TYPES OF TESTING

The test plan should detail the types of testing to be conducted, such as **unit, integration, system, UAT (User Acceptance Testing)**, and **performance testing**.



Test Execution Methodology

A test execution methodology should emphasize a **structured approach** to executing tests within Salesforce environments, ensuring **comprehensive coverage** and **alignment with best practices**.

❖ ACCEPTANCE CRITERIA

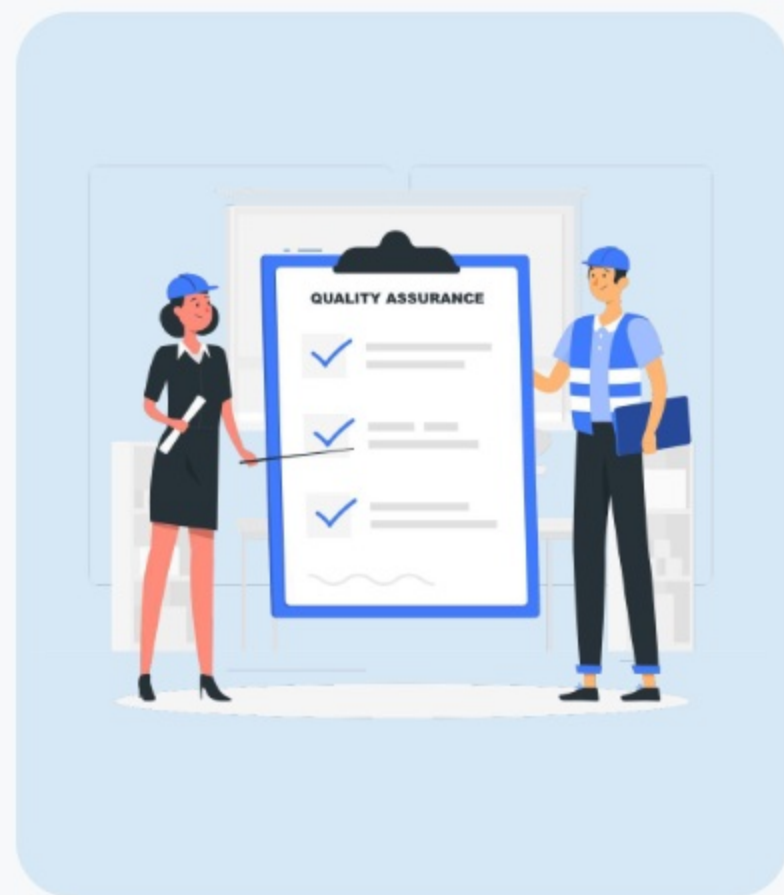
To **minimize the number of defects**, it is necessary to ensure that the **acceptance criteria** for tests are **detailed**.

❖ COVERAGE

A test plan should focus on **code coverage**, coverage of **features and business processes**, and **security and compliance coverage**.

❖ AUTOMATION

Salesforce DX and **CI/CD tools** should be used whenever possible to automate test execution and deployment processes.



Test Execution Methodology

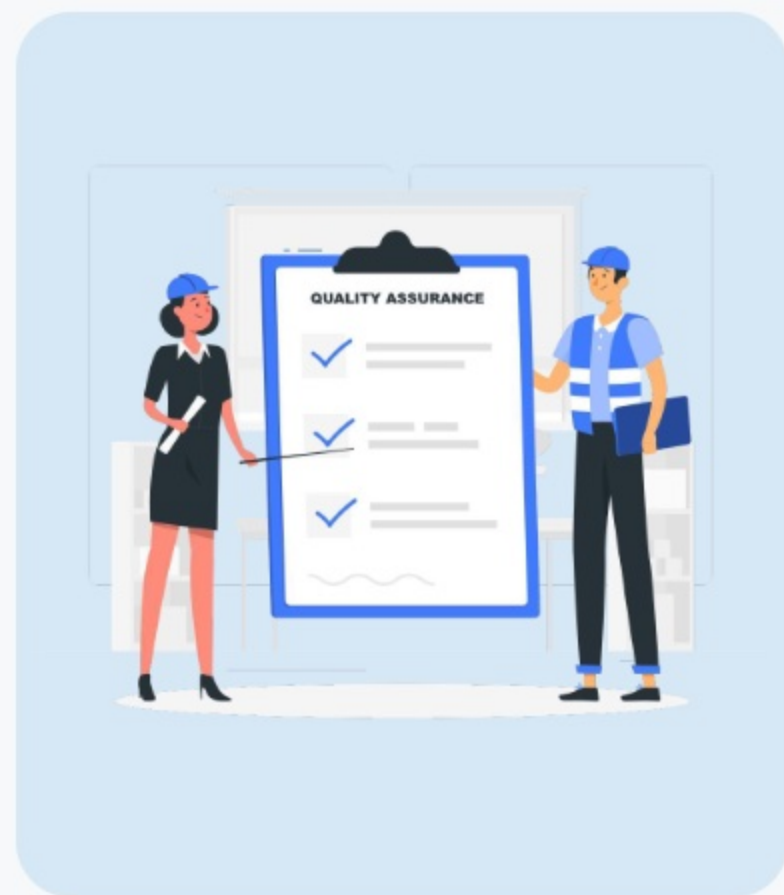
A test execution methodology should emphasize a **structured approach** to executing tests within Salesforce environments, ensuring **comprehensive coverage** and **alignment with best practices**.

❖ MONITORING

Test results should be **reviewed** regularly, and tests should be **adjusted** as business processes and functionalities evolve.

❖ STAKEHOLDERS

Business stakeholders should be kept **engaged** in the testing process, especially during **UAT**, to ensure the solution meets business needs.



Apex Test Execution

Table of Contents



Executing Apex Unit Tests

Running Apex unit tests involves **creating test data**, **calling the test methods**, and then **verifying the results**. Data is **not committed**, so it does not need to be **deleted**.



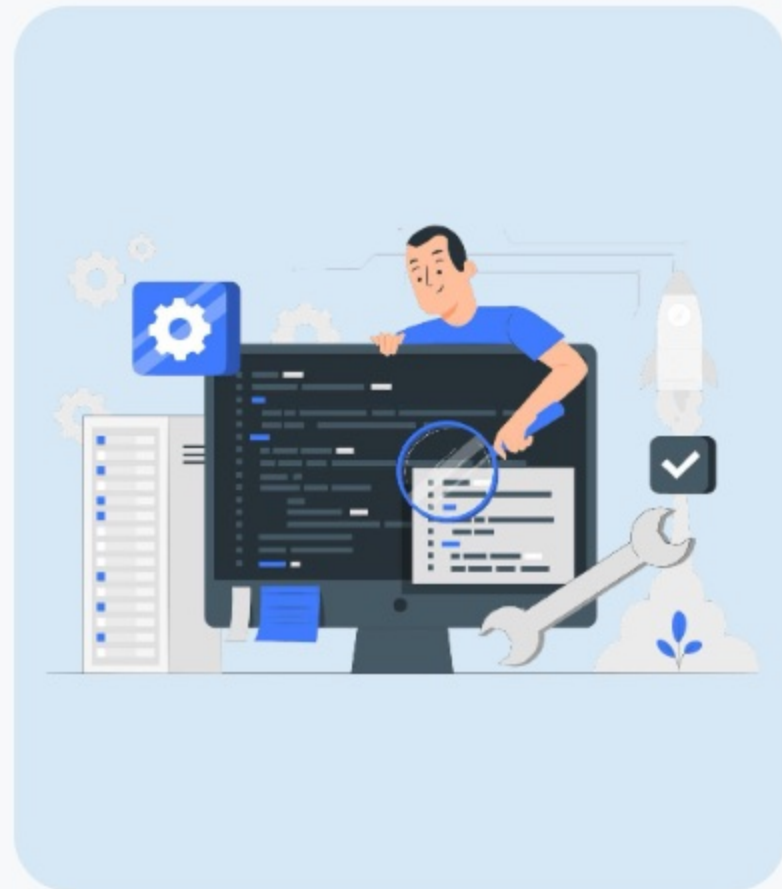
Executing Apex Unit Tests

Various **options and considerations** apply to the execution of **Apex unit tests** in Salesforce.

❖ TOOLS

Apex unit tests can be **run using these tools**:

- ❖ Apex Test Execution Page
- ❖ Apex Classes Page (Run All Tests)
- ❖ Developer Console
- ❖ Visual Studio Code
- ❖ Code Builder
- ❖ API (SOAP API , Tooling REST API, etc.)



Executing Apex Unit Tests

Various **options and considerations** apply to the execution of **Apex unit tests** in Salesforce.

❖ GROUPINGS

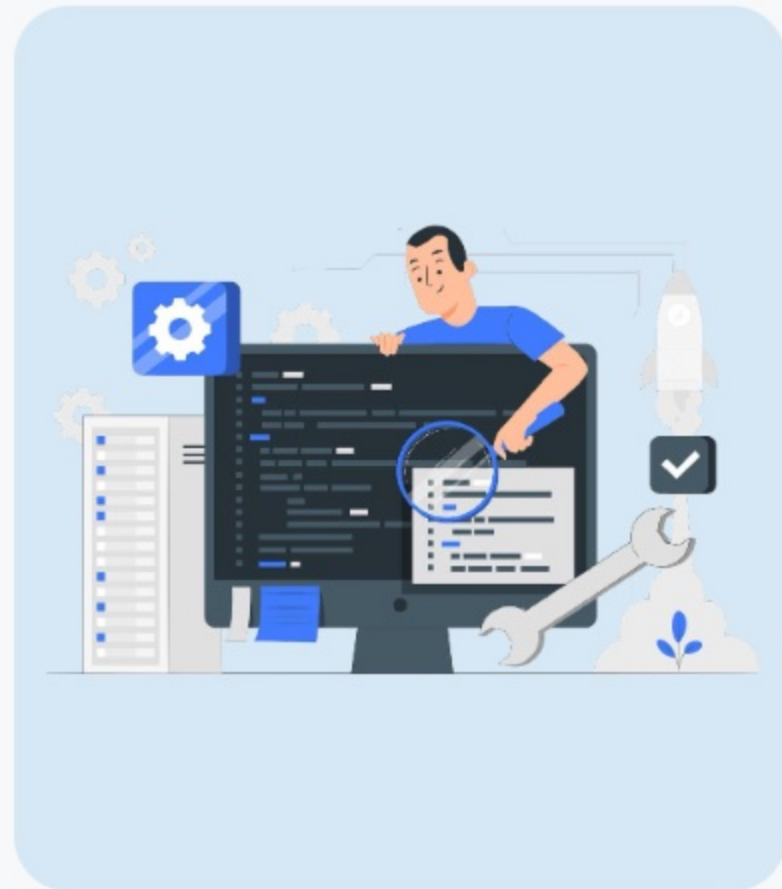
Salesforce allows **executing these groupings of unit tests**:

- ❖ Some or all methods in a test class
- ❖ A predefined suite of test classes
- ❖ Some or all methods in a set of test classes
- ❖ All unit tests in an org

❖ TEST OPTIONS

When **validating or deploying** a change set, **four test options** are available:

- ❖ Default
- ❖ Run Local Tests
- ❖ Run All Tests
- ❖ Run Specified Tests



Executing Apex Unit Tests

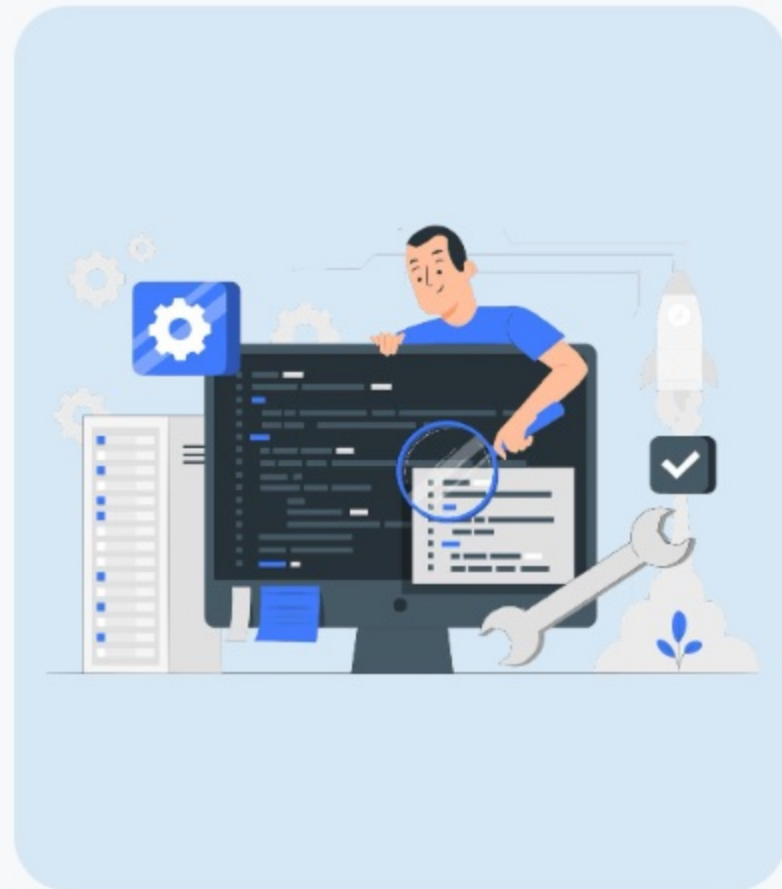
Various **options and considerations** apply to the execution of **Apex unit tests** in Salesforce.

❖ SUBSET OF TESTS

When using Metadata API for a deployment, the **RunSpecifiedTests** test level can be set on the **DeployOptions** object to run a subset of tests and **shorten the deployment time** to production. Each **test class** (and **not individual test methods**) needs to be specified in the DeployOptions object, which is then passed as an argument to the **deploy()** call.

❖ DEPLOYMENT REQUIREMENT

A deployment fails if the **code coverage** of an Apex component in the deployment is **less than 75%**. It also fails if **one of the specified tests fails**. Code coverage is computed **individually** for each Apex **class** and **trigger** and is different from the overall code coverage percentage.



Executing Apex Unit Tests

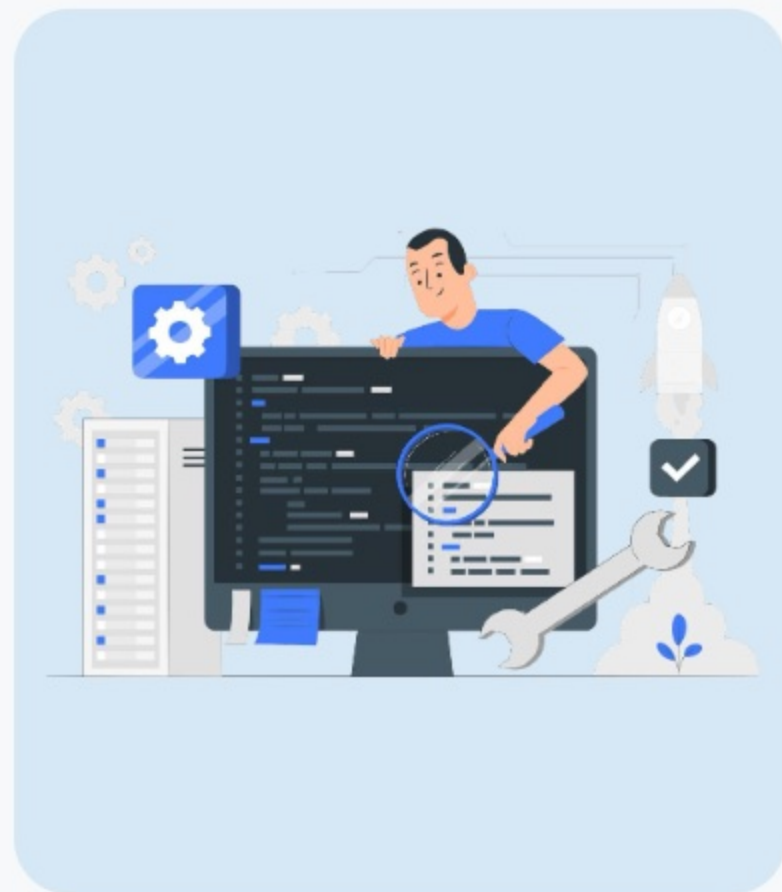
Various **options and considerations** apply to the execution of **Apex unit tests** in Salesforce.

❖ OUTBOUND EMAIL

Apex unit tests **cannot send** outbound emails. When a unit test includes Apex code that sends an email message, it may run successfully, but **no email** is actually sent out from the org.

❖ APEX CALLOUTS

Apex unit tests **cannot perform** HTTP callouts. In order to test Apex code that performs callouts, a **mock object** needs to be created by defining a class that implements the **HttpCalloutMock** interface. The **Test.setMock()** method is used to set the mock object.



Executing Apex Unit Tests

Various **options and considerations** apply to the execution of **Apex unit tests** in Salesforce.

❖ LIMITS METHODS

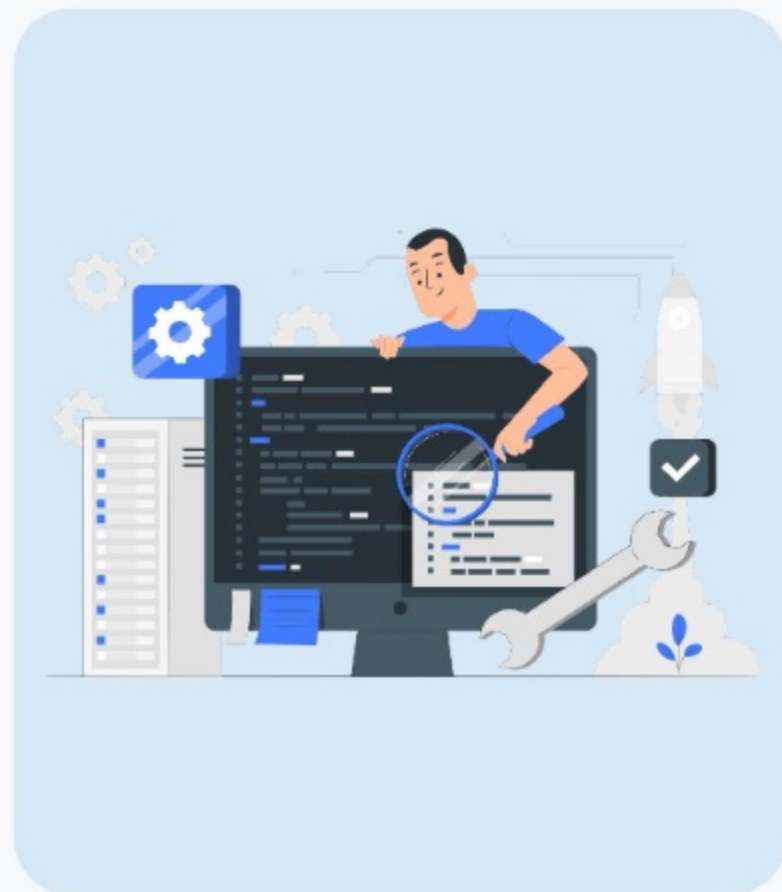
The Limits methods can be used to return the **specific limit for a particular governor**, such as the number of callouts that have been processed or the amount of heap size remaining.

❖ RESETTNG LIMITS

The **startTest()** and **stopTest()** methods can be used to reset governor limits in Apex unit tests. A new set of governor limits is assigned to any code that executes after startTest() and before stopTest().

❖ PRODUCTION DATA

It is a best practice to **not use the @IsTest(SeeAllData=true) annotation** since the test results can differ depending on which data is available in the organization. Unit tests should **generate their own test data**.



Using a Mock Object

The following shows an example of a **mock object** and how it is used in a **test method**.

```
// A mock class that simulates the response of a successful enrollment of a student:
public class EnrollHttpCalloutMock implements HttpCalloutMock {
    public HttpResponseMessage respond(HttpRequest req) {
        HttpResponseMessage res = new HttpResponseMessage();
        res.setHeader('Content-Type', 'application/json');
        res.setBody('{ "success":true,"studentId":12345,"courseId":67,"yearLevel":3}'); // create fake response
        res.setStatusCode(200);
        return res;
    }
}

// Example code contained and executed inside a test method:
Test.startTest();
Test.setMock(HttpCalloutMock.class, new EnrollHttpCalloutMock());
StudentApplicationWrapper application = generateStudentApplication(); // some helper method
Enrollment erm = StudentController.enroll(application); // invoke the method to test

// verify new record created from fake data
System.assertEquals(erm.ExternalStudentId__c, 12345);
System.assertEquals(erm.YearLevel__c, 3);
System.assertEquals(erm.CourseId__c, 67);
Test.stopTest();
```

Test.setMock() should be called after **Test.startTest()** and before the callout. The **mock object** is passed as an argument.

Code Coverage

[Table of Contents](#) 

Code Coverage

Code coverage numbers are generated for Apex code every time **one or more tests are executed**.

❖ COVERED APEX CODE

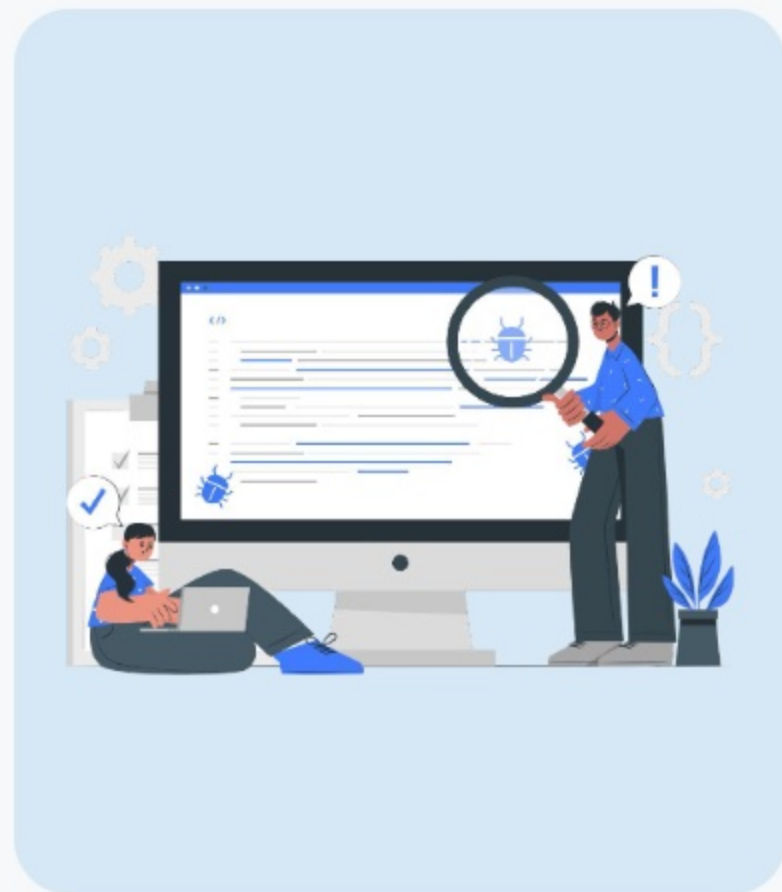
Code coverage indicates the number of **executable lines of code in Apex classes and triggers** that have been exercised by test methods.

❖ COVERAGE REQUIREMENTS

Apex unit tests enable meeting the code coverage requirements for **deploying** or **packaging Apex code**. The tests must cover **at least 75%** of the **Apex code** and **pass**, **every trigger** must have **some test coverage**, and **all classes and triggers** must **compile** successfully.

❖ CODE QUALITY

Code coverage **does not guarantee test effectiveness**. Unit tests should **assert all the expected behaviors** and **ensure code quality**.



Code Coverage

Code coverage numbers are generated for Apex code every time **one or more tests are executed**.

❖ INSPECTION

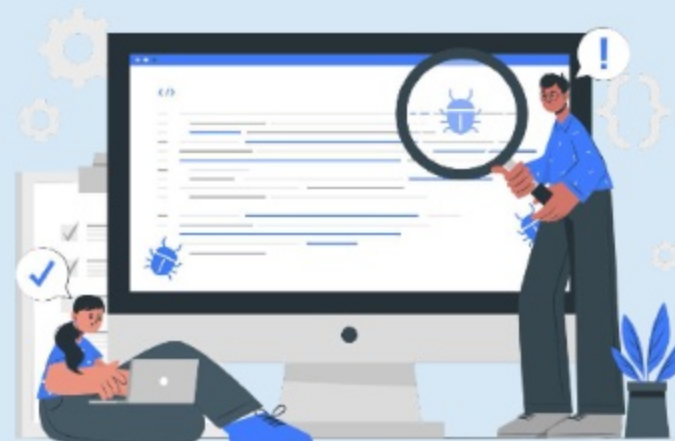
Code coverage information can be viewed in the **Tests** tab of the **Developer Console**.

❖ REFRESH

Tests need to be **rerun** to **refresh code coverage numbers**. They aren't refreshed automatically when Apex code is updated.

❖ DISCREPANCIES

There can be **discrepancies in code coverage numbers** between a **development or sandbox environment and production** due to reasons such as test failures, data dependencies, metadata dependencies, etc.



Code Coverage

Code Coverage Percentage, which is computed when running tests, uses the formula:

$$CCP = [(number\ of\ covered\ lines) / (number\ of\ covered\ lines + number\ of\ uncovered\ lines)] * 100$$

The following are **not counted** and **do not impact** code coverage percentage:

- ❖ Code comments
- ❖ Class name definition
- ❖ Lines that only contain brackets
- ❖ *System.debug* statements

Only the lines **highlighted in blue** are included in the code coverage computation.

```
1 // this is a class used for controlling a vehicle
2 public class Vehicle {
3
4     static Integer maxCapacity = 4;
5
6     // add passengers to vehicle
7     public static void addPassengers(List<Contact> contacts) {
8         System.debug('passenger count: ' + contacts.size());
9         if (contacts.size() > maxCapacity) {
10             System.debug('Too many passengers!');
11         } else {
12             notifyPassengers(contacts);
13         }
14     }
15
16     // send email to vehicle passengers
17     public static void notifyPassengers(List<Contact> contacts) {
18         System.debug('alert passengers..');
19     }
20 }
```


Testing Lightning Components

[Table of Contents](#) 

Testing Lightning Web Components

Jest is a **testing framework** that can be used to write unit tests for **Lightning web components**.

❖ RUNNING JEST TESTS

Jest tests can be run in the **command line** or the **IDE**. They **don't run in the browser** or **connect to an org**, and they give **immediate feedback** while coding when run in **watch mode**.

❖ TEST CASES

Jest tests can be written to test a **component in isolation**, its **public API**, **basic user interaction (clicks)**, **DOM output**, **firing of events**, etc.

❖ USING JEST

Jest and its dependencies can be installed in a **Salesforce DX** project. Jest tests can be written in **local JavaScript files**. They can be run from **VS Code** or the **command line**.



Testing Lightning Web Components

Jest is a **testing framework** that can be used to write unit tests for **Lightning web components**.

❖ TEST FILE

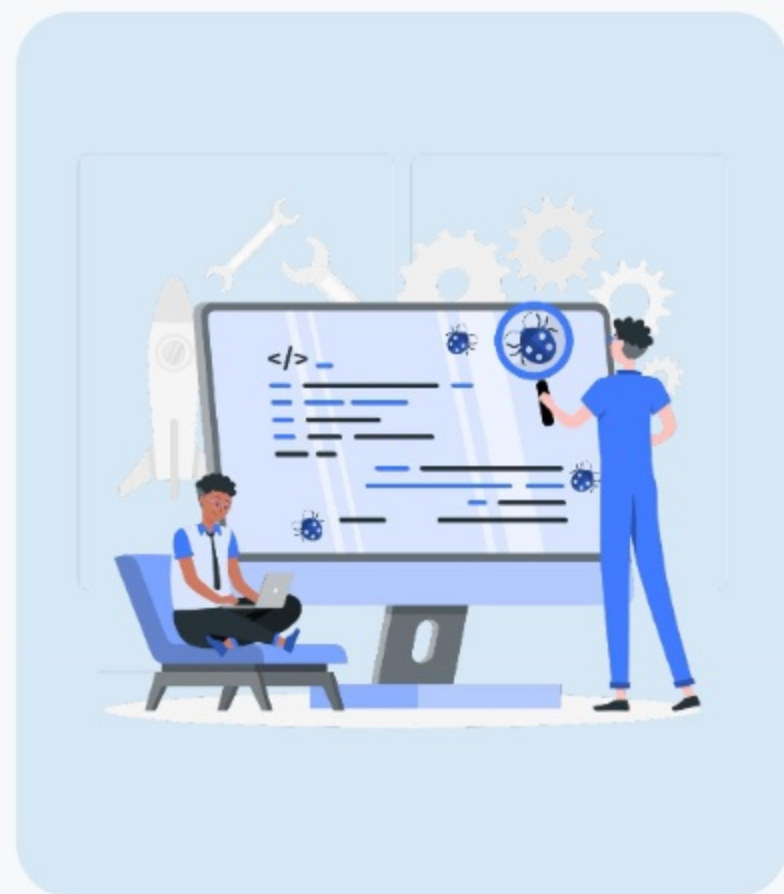
Each test file must import both the **component** to **test** and the **createElement** method from **'lwc'**. A describe block should be **defined** to create a **test suite** within the test file.

❖ TEST BLOCKS

An **it** or **test** block should be defined to create a single functional **unit test**. The DOM should be **reset** at the end of each unit test using the **afterEach()** method.

❖ CREATE INSTANCE

Inside the unit test, the imported **createElement** method can be used to create an **instance** of the component for testing. It should be **appended** to the body of the document using the **appendChild()** method.



Testing Lightning Web Components

Jest is a **testing framework** that can be used to write unit tests for **Lightning web components**.

❖ SHADOWROOT

When creating a unit test, the **shadowRoot** property of the element under test should be used to access the shadow tree. It can be used to select a particular element.

❖ PROMISES

Asynchronous DOM updates such as a property change can be tested by using a **Promise** in the unit test.

❖ OTHER CONSIDERATIONS

To test the wire service, the **@salesforce/sfdx-lwc-jest test utility** can be utilized. A JSON file can be **created** to define the Salesforce data **expected** by a component. The **npm run test:unit:watch** command can be used to **run tests continuously** during development.



Testing Aura Components

One of the testing frameworks that can be used to perform automated testing of Aura components.

❖ TESTING TOOLS

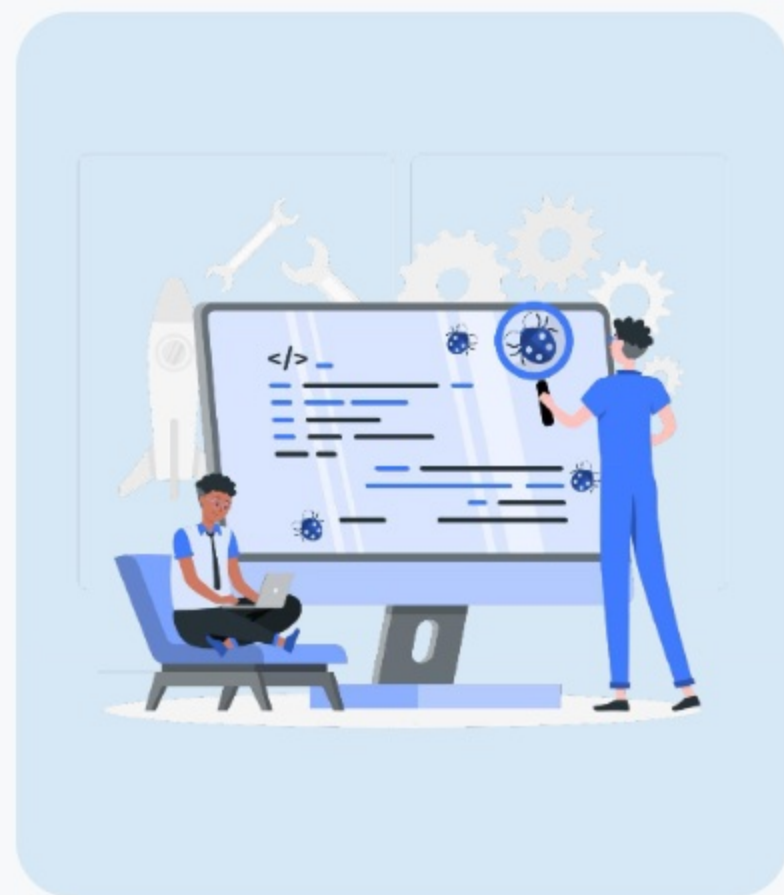
Some popular testing tools that can be used to test Aura components are **Jest, UTAM, Jasmine, Mocha, Selenium, and WebdriverIO**.

❖ ACCESSIBILITY TESTS

Salesforce's test tools can be utilized to check for common accessibility issues (such as **images without alt** attributes and **th element without a scope** attribute) in Aura components. **Aura accessibility tests** can be called in **two environments: JavaScript and WebDriver**

❖ APEX CONTROLLER

Server-side logic in the component's **Apex controller** must be tested separately using an **Apex test class**.



Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Web Solutions manages its customer relationships and operations using separate Salesforce instances for its branches in Asia and South America. The company has encountered difficulties in maintaining a consistent quality assurance process, leading to disparities in customer experience and operational efficiency between the regions. Recognizing the need for better testing and quality control methodologies, the company turns to a Salesforce architect for guidance.

The architect is charged with evaluating the existing testing strategies employed by the two divisions and suggesting appropriate test execution methodologies and coverage needs for the company's requirements. The goal is to establish consistent, high-quality testing practices across both divisions, enhancing the reliability of deployments.

Requirements & Solutions

[Table of Contents](#) 

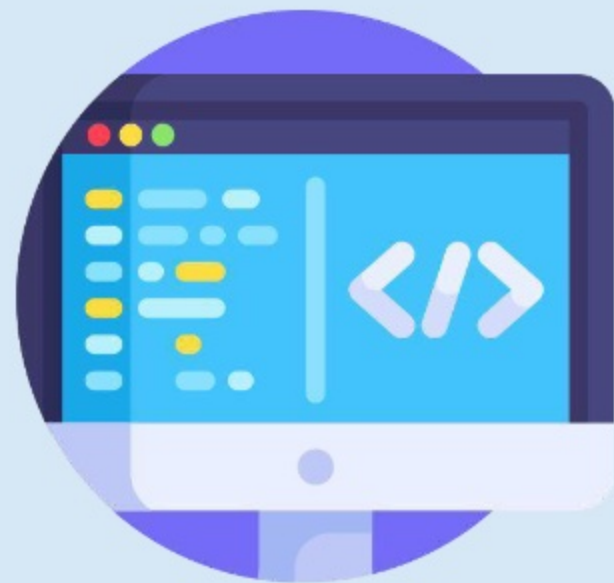
Requirement 1 (Part 1)



SCENARIO

The development team of the company's Asian division has certain requirements related to testing and quality assurance:

- 1) The Salesforce instance has 85% overall code coverage. A test plan is created for each sprint, but the number of defects is still large. The architect needs to suggest a suitable approach to reduce the number of defects.
- 2) Automated browser tests are used to verify the functionality of Lightning web components. The team is concerned about the impact of Salesforce releases and needs to identify the type of changes that would require updating the tests.

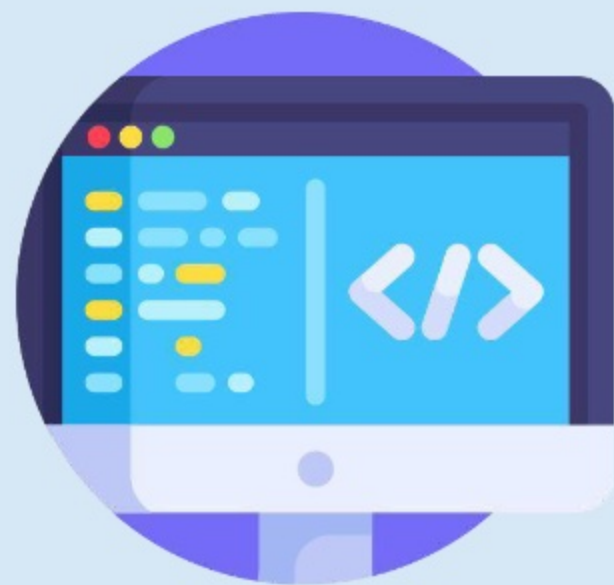


Requirement 1 (Part 2)



SCENARIO

- 3) A developer has written a test class to test a method that involves a web service callout. However, the test method fails and this error message is returned: *Methods defined as TestMethod do not support Web service callouts*
- 4) The team has written test classes to unit test certain Apex classes and triggers. When executed in the sandbox, all the test methods pass and all the classes meet the minimum code coverage requirement. But a few of these test methods are failing when deploying the associated Apex components to production.



Solution 1 (Part 1)



SOLUTION

- 1) To reduce the number of defects, **code coverage should be increased** and the **acceptance criteria** should be **more detailed**.
- 2) Any **DOM changes** introduced in a Salesforce release could require updating the automated browser tests.
- 3) To avoid the error message, a **mock object** should be used to test the web service callout. The following steps should be performed within the test class:
 - a) Create an Apex class that implements the **HttpCalloutMock interface**.
 - b) In the test method, call the **Test.startTest()** method.



Solution 1 (Part 2)



SOLUTION

- c) Set the mock object using the **Test.setMock()** method.
 - d) Call the Apex method that contains the **web service callout**.
 - e) **Verify** the behaviors using assertions.
 - f) Call the **Test.stopTest()** method.
- 4) The test methods are most likely failing because of the **@IsTest(SeeAllData=true)** annotation and the presence of different data in production. Test methods should **not use this annotation** to ensure that test data is not dependent on the org's data . They should **generate their own test data**.

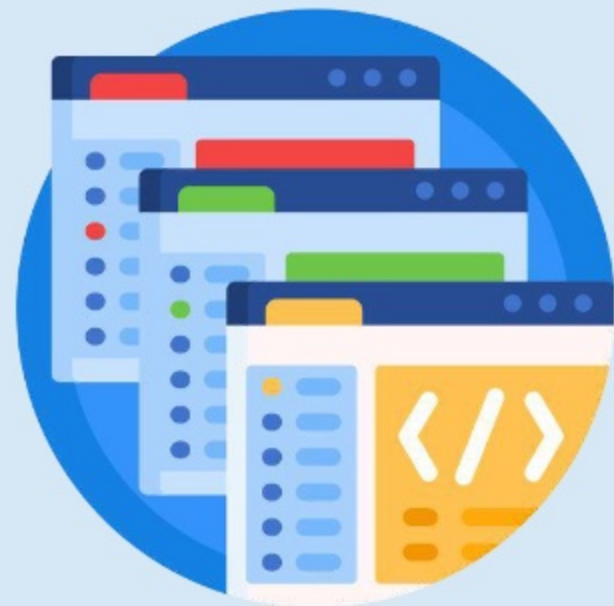


Requirement 2



SCENARIO

The development team in South America is facing certain challenges related to testing. The developers have built new features for the sales team, but while testing the Apex classes, they are constantly hitting the governor limits. Various Aura components are utilized by these new features, but the team is unsure how to test these components. They also need to perform accessibility tests for these Aura components. In addition, the team lead would like to shorten the deployment time to production and would like considerations related to running specific tests during deployments using Metadata API.



Solution 1 (Part 1)



SOLUTION

While testing the Apex classes, the governor limits can be reset using the `Test.startTest()` and `Test.stopTest()` methods. Apex methods that need to be tested should be called after `Test.startTest()` and before `Test.stopTest()`.

The JavaScript code in Aura components can be tested using a suitable **testing framework such as Jest**. Server-side logic in Apex controllers should be tested using **Apex test classes**. Salesforce's test tools can be utilized to check for common accessibility issues in Aura components. **Aura accessibility tests** can be called in two environments: **JavaScript** and **WebDriver**



Solution 1 (Part 2)



SOLUTION

The deployment time to production can be shortened by running a subset of tests. The **RunSpecifiedTests** test level can be set on the **DeployOptions** object. The **test classes** to be executed should be specified in `DeployOptions`; **specifying test methods is not supported**. The object should be passed as an argument to the **deploy()** call.



Learn More

- [!\[\]\(1207edb9a08751d3d55970560645ed23_img.jpg\) Testing Apex](#)
- [!\[\]\(d7a34a706cfa4ef37c62a369101e1b36_img.jpg\) Testing Lightning Web Components](#)
- [!\[\]\(7325769475e8f4bf67f57a0cbebc8ab9_img.jpg\) Testing Components](#)
- [!\[\]\(1a468f12cdfc63dc07896d0781cf55ec_img.jpg\) Running a Subset of Tests in a Deployment](#)
- [!\[\]\(a9a0baec8ceb7d7c04180806eca8d32a_img.jpg\) Apex Testing](#)
- [!\[\]\(c1ab807d6aebb565b3082513037b5622_img.jpg\) Advanced Apex Testing](#)
- [!\[\]\(083bb479299cb5e55cd99db0433ca6ba_img.jpg\) Lightning Web Components Tests](#)

